

Cloud handoff — #496 + #365, the fixture catalogue finished and version-proofed

Written: 2026-08-25 · **By:** max (CLI session on Tom's box) · **For:** a cloud Claude Code session on `tom2025b/git-vista`

Two jobs, one theme: **#448's catalogue is one builder short of true, and its parser has only ever been exercised against one git version.** The first is a twenty-minute move. The second is the reason this went to the cloud rather than staying here — provisioning a second git binary is trivial in a container and awkward on Tom's box.

```
task_id: gv-fixtures-finished
issues: [496, 365]
milestone: -
repo: tom2025b/git-vista
base: main
branch: fix/496-365-fixtures-finished
sign_commits_as:
  name: Claude_Max
  email: 262510778+tom2025b@users.noreply.github.com
sign_artifacts_as: max
adr_number: 0082          # ASSIGNED, for #365's version-floor deci
                        # #496 is mechanical and needs none.
allowed_paths:
  - crates/git-vista-fixtures/**
  - crates/git-vista-protocol/src/status.rs  # the comment at 258-
  - ci/browser/fixture.mjs
  - ci/browser/global-setup.mjs
  - .github/workflows/ci.yml
  - docs/SUPPORTED_VERSIONS.md
  - docs/adr/
forbidden_paths:
  - design-docs/
  - crates/git-vista/src/**
  - crates/git-vista-server/src/**
  - handoff.md
merge_order: independent of the other five. Nothing else touches th
```

Part 1 — #496: the one builder #448 did not move

`crates/git-vista-fixtures/` already holds `browser.rs`, `seeded.rs`, `conflict.rs`, `broken.rs`, `content.rs` and `git.rs`. Every browser fixture shape lives there and is built by the `gv-fixture` binary — **except the stash repository**, which `ci/browser/fixture.mjs` still builds in JavaScript.

How it got there, and why it is worth reading before you start. #448 and PR

490 (#77, the stash drawer) were in flight at the same time. #448 moved every

builder into Rust and correctly deleted what only they used — the `node:fs` imports and the `IDENT` git-identity constant. #490 landed `buildStashFixture` in that same file and still needs all of it.

Neither branch touched the other's lines, so git merged them cleanly and the merged file called `rmSync` and spread `IDENT` with neither in scope:

```
ReferenceError: rmSync is not defined
ReferenceError: IDENT is not defined
```

A clean text merge and a broken program. It surfaced only when the browser harness actually ran — a leg neither PR's CI runs and no cloud session can execute. Both were restored on the merge (`8504ca80`) with a comment saying they are on loan.

Two things this makes true that are worth stating in the ADR-less commit message rather than losing:

1. **448's claim is *one* catalogue whose doc comments are the teaching material,**

so a lesson that drifts from the code fails a test. That claim is currently false by one builder, and the exception is invisible from the Rust side — exactly where a reader would go to confirm it.

2. **IDENT is not cosmetic.** It pins the commit identity per invocation because this box has repositories whose local `user.email` is a personal gmail address, and a bare `git commit` picks it up silently. A JavaScript builder keeping its own copy of that rule is a second place for it to be wrong.

Part 1 acceptance

- The stash fixture is built by `gv-fixture`, like every other shape, with its reasoning in the Rust doc comments — including *why* the fixture's stash subject deliberately collides with its seed commit's subject, which is what `openDrawer`'s scoped locators exist to survive.
- `ci/browser/fixture.mjs` needs neither `node:fs` nor `IDENT` afterwards, and **both are deleted with the builder** rather than left behind.
- The four stash browser tests still pass. **You cannot run them** — see below.

Part 2 — #365: the parser has met exactly one version of git

`crates/git-vista-protocol/src/status.rs` says so itself, at lines 258-276, and says it honestly rather than claiming the criterion satisfied:

...this parser has only been *exercised* against 2.43.0 — a single version, not the floor itself. That is a real gap against #68's "*results match supported Git versions on fixtures*" criterion, named here rather than silently claimed satisfied.

The floor is **2.32**, and `docs/SUPPORTED_VERSIONS.md` derives it from something real: `GIT_CONFIG_GLOBAL` was added in 2.32, and the fixture harness depends on it. `.github/workflows/ci.yml` (around line 157) parses that floor out of the document and **rejects a runner older than it** — but it does not provision the floor, and it does not test against it. So the check enforces "not older than 2.32" while every fixture has only ever seen 2.43.0.

This is the half of the pair the cloud is genuinely better at. Building or fetching a git 2.32 binary in a container is routine; doing it on Tom's box means installing a second git next to the one he uses daily, which is a change to his machine for a test's benefit.

What the ADR has to decide

Not "should we test the floor" — the issue settles that. The decision worth recording as **ADR 0082** is *how the floor stays honest as it moves*:

- **Where does the 2.32 binary come from?** A distro package, a build from source, or a pinned container image. Say which and why, and what happens when that source disappears — a version-floor job that silently stops running is worse than none, because the comment in `status.rs` would then be wrong in the reassuring direction.
- **The floor job is mandatory, not a choice for the ADR.** #365's own acceptance criteria already settle this: "Make the 2.32 leg mandatory rather than silently falling back to the runner's ambient Git." What the ADR actually decides is how mandatory survives a *transient* failure fetching an eleven-year-old git — retry, a cached/pinned artifact, or a pinned container image — so a flaky fetch neither blocks every merge on infrastructure noise nor becomes the excuse to quietly downgrade to advisory.
- **What happens when `SUPPORTED_VERSIONS.md` changes the floor?** The existing CI step already parses the document rather than hardcoding — keep that property. The new job must read the same source, so the doc and the test can never drift apart.

Part 2 acceptance

- A real git 2.32 binary is provisioned reproducibly, alongside the normal current-git leg.
- Real fixture repositories are built and `git status --porcelain=v2 --branch -z` is run with **both** binaries.
- The #68 vocabulary is exercised on both: staged, unstaged, both-sides dirty, rename/copy (the two-token `-z` split), untracked, ignored, conflict, and submodule shapes.
- Both versions' real output is parsed through `parse_porcelain_v2_z` and the results compared. **A difference is a finding, not a failure to paper over** — if 2.32 genuinely produces a different shape, that is the thing this issue exists to discover, and it belongs in the PR body in its own section.

- **The comment at `status.rs:258-276` is updated to say what is now true.** Leaving an honest admission in place after closing the gap it admits is its own small lie.
 - **`docs/SUPPORTED_VERSIONS.md` documents the local reproduction command** — #365's acceptance criteria require this explicitly ("Document the local reproduction command") and it is currently missing from this list even though the file is already in `allowed_paths`.
-

What you cannot run, and what to do instead

The browser leg does not run in a cloud container. The server refuses to start without its strict sandbox tier, and the kernel there reports `landlock_abi=-1`; INV-13 gives it no degraded mode. Installing `bwrap` changes nothing — it is the **kernel's** missing capability, not the container's. Two sessions hit this independently on 2026-08-25.

Part 1 is *entirely* about a fixture the browser harness consumes, so this matters more here than in any of the other five handoffs:

- Build the fixture with `gv-fixture` and **inspect the resulting repository with plain git** — `git stash list`, `git log --oneline`, `git status --porcelain` — and put that output in the PR body. That is the closest you can get to proof, and it is a lot closer than nothing.
- **Say explicitly in the PR body that `ci/browser/run.sh` is unrun.** Do not leave it implicit. A session on Tom's box will run it before merge.

`cargo test --workspace` is yours and must be green. Two tests in `git-vista-server` flake under parallel execution because they race on the process-global current repository (#438):

`recovery_center::tests::a_stale_claimed_undo_is_refused_and_th`

`e_branch_is_left_alone` and

`state::tests::selection_flow_carries_mode_and_gates_writes`. Re-run before believing either is your doing.

Acceptance, both parts

1. Part 1 and Part 2 acceptance above, in full.
 2. Every test you add that pins an invariant is proved able to go red **two different ways** — remove the mechanism, and weaken it. One `caught` verdict is not proof: a Git-Vista test survived one mutation and caught another on 2026-08-22, and either alone gives the wrong verdict.
 3. **ADR 0082** records the version-floor decision.
`docs/adr/README.md` index updated.
 4. `cargo fmt --all`, `cargo clippy --all-targets -- -D warnings`, `cargo test --workspace` green.
 5. PR body says `Closes #496` and `Closes #365`, and states that the browser leg is unrun. **Never delete the branch.**
-

Signed: max · 2026-08-25T07:20:00-04:00